Universidad de Antioquia

Facultad de Ingeniería

Informática II

DESAFÍO 2

ude@Book red social implementada con programación orientada a objetos en c++

Estudiantes

Juan Esteban López Castrillon

Juan Camilo López Castrillon

Profesor

Aníbal Guerra

Semestre 2026-2

Medellín, 18-10-2026

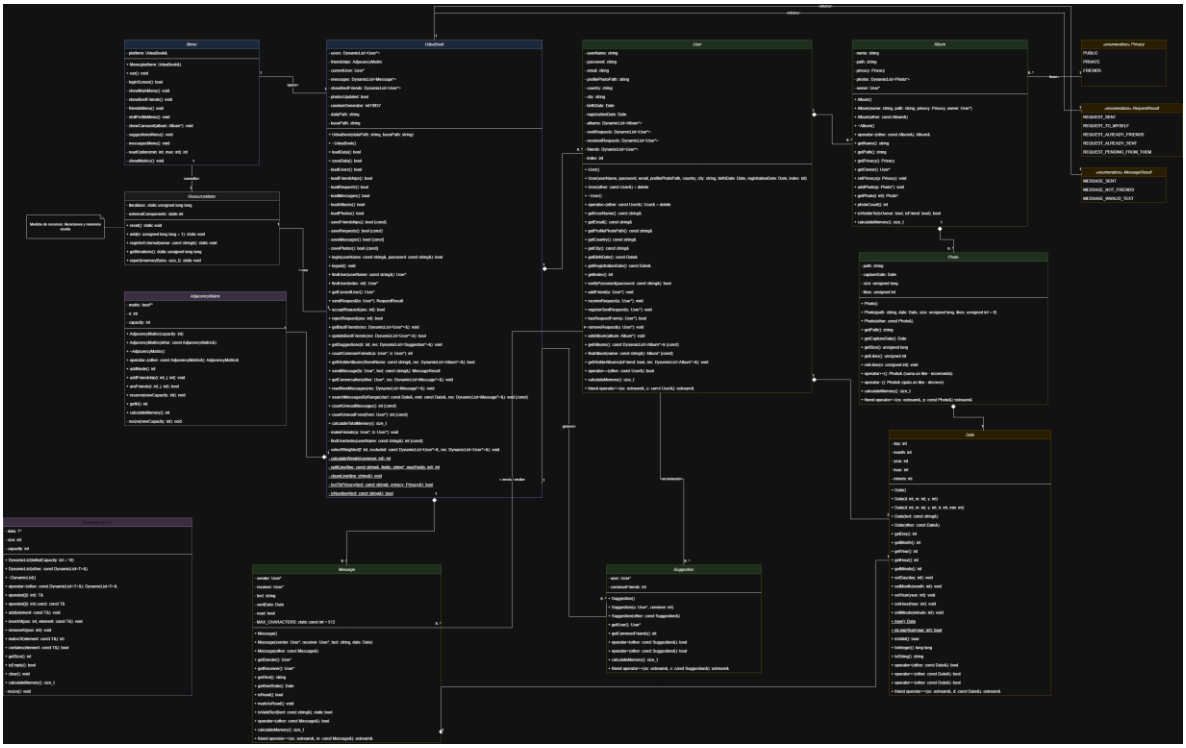
Análisis del problema.

Ude@Book es una red social por consola, en donde los usuarios pueden iniciar sesión, administrar sus amistades, ver sugerencias de amistad, visitar perfiles y en esos perfiles poder ver sus álbumes de fotos si la privacidad de los álbumes es publica o de amigos, tambien pueden enviarse mensajes, la información se guarda en archivos de texto que es como nuestra base de datos y esta se carga al iniciar, y hay que medir cuanta memoria e iteraciones usa cada funcionalidad para medir su eficiencia.

Este desafío estaba planteado para que desarrolláramos la solución usando programación orientada a objetos por lo que de manera abstracta desarrollamos y planificamos cada entidad o clase, primero hicimos el diagrama de clases o UML el cual nos permitía modelar cada clase que íbamos a utilizar y que atributos y métodos necesitábamos para cada clase de acuerdo a su funcionalidad, tuvimos en cuenta mucho la utilización de referencias para evitar copias innecesarias y así ser más eficiente en la utilización de la memoria del programa.

En el desarrollo del programa nos encontramos con varios problemas por ejemplo al no ser permitida la utilización de estructuras o la librería stl nos vimos en la necesidad de realizar nuestras propias estructuras de datos para facilitarnos la implementación de algunas funcionalidades, por ejemplo, en el desarrollo creamos una plantilla o por así decirlo una estructura de datos que nos permitía almacenar de manera dinámica algunos de nuestros objetos, por ejemplo, las fotos, álbumes etc. Esa plantilla la llamamos DynamicList y para las amistades hicimos AdjacencyMatrix.

Diagrama de clases.



Algoritmos esenciales

Búsqueda de usuarios con búsqueda binaria

Como users.txt está ordenado alfabéticamente, en cada paso se compara el usuario buscado con el de la mitad del rango y se descarta la mitad donde no puede estar, así con 1000 usuarios bastan unos 10 pasos en vez de 1000. La mitad se calcula como low + (high − low) / 2 para evitar desbordamientos.

Sugerencias de amistad

Se recorren los amigos de cada amigo del usuario y se cuenta cuántas veces aparece cada persona que todavía no es su amigo, ese conteo es el número de amigos en común. Para mostrar las K mejores no se ordena toda la lista, solo se busca el mejor candidato K veces usando el operador > sobrecargado en Suggestion.

Mejores amigos con prioridad

Cada amigo recibe un peso según los amigos en común, 1 si son hasta 10, 2.5 si son entre 11 y 20 y 3.5 si son más de 20, guardados como 2, 5 y 7 para usar solo enteros. Se sortea un número entre 0 y la suma de los pesos menos 1 y se acumulan los pesos hasta superarlo, así quien tiene más peso tiene más probabilidad de salir. El elegido queda con peso cero para no repetirse, y al actualizar la lista, que solo se puede una vez por sesión, también se excluyen los que ya se mostraron.

Carga de archivos

Cada línea pasa por cleanLine, que quita el \r de los archivos de Windows, y luego por splitLine, que la parte por | y descarta las líneas con una cantidad de campos incorrecta, conservando completo el texto de los mensajes aunque tenga |. Cada amistad se guarda en la matriz de adyacencia para consultas rápidas y en la lista de amigos de ambos usuarios.

Carrusel de fotos

Para avanzar se usa (posición + 1) % total y para retroceder (posición − 1 + total) % total, sumando total para que el resultado nunca sea negativo, así de la última foto se pasa a la primera y viceversa sin salirse del arreglo.

Problemas de desarrollo

Diseño y memoria

Lo primero fue decidir quién borraba qué, porque si dos partes del programa borran lo mismo fallaba. Por ejemplo, con los mensajes teníamos ese problema, porque son de dos usuarios al tiempo, entonces al final le dejamos ese trabajo a el objeto UdeaBook. Los álbumes sí quedaron a cargo de cada usuario, y por eso tocó prohibir que se pudiera copiar un usuario.

En el programa no usamos copias de ningún álbum, pero el enunciado pide usar constructores de copia, entonces hicimos un constructor de copia profunda para

evitar posibles problemas. Si alguien hace una copia de un álbum, en las fotos se copian las direcciones de memoria, porque eso es lo que estamos almacenando, entonces si se elimina el original o la copia, el otro queda con las direcciones de fotos que ya no existen y al eliminarse las borra otra vez. Por eso cada álbum debería tener sus propias fotos, así sean una copia.

Otro problema que enfrentamos es que queríamos saber si dos personas son amigas, pero también poder recorrer solo los amigos de alguien sin pasar por los mil usuarios. Lo que hicimos fueron las 2 cosas, una matriz para preguntar rápido y listas para recorrer solo los amigos de verdad.

Y como cada amistad y cada solicitud queda guardada en los dos usuarios, al guardar en el archivo salían repetidas. Lo arreglamos guardándolas solo desde un lado con el getIndex() > i.

Otro error que presentamos en el código fue con DynamicList: la habíamos definido con paréntesis en vez de usar corchetes, entonces solo creaba un elemento en vez de un arreglo. Luego nos dimos cuenta y lo solucionamos.

Archivos y datos

Los archivos de Windows meten un \r escondido al final de cada línea, y se nos quedaba pegado al último dato entonces hicimos cleanLine, que lo limpia.

Trabajo en equipo

Otro problema que enfrentamos fue que a veces teníamos conflictos en Git al momento de hacer merge o de traer una rama a una rama propia. Como cada uno agregaba sus archivos al CMakeLists.txt, en cada merge había conflictos justo en ese archivo, pero uno solucionaba los conflictos y quedaba todo bien.

Evolución de la solución

En la etapa de clases base se desarrollaron primero DynamicList, Photo, Privacy y Album, porque las demás clases dependían de ellas para guardar sus datos. Por ejemplo, Album necesitaba la DynamicList para almacenar sus fotos. En Album también se implementó la copia profunda, para que al copiar un álbum se dupliquen sus fotos y no queden dos álbumes compartiendo la misma información en memoria. Después se desarrollaron Date y AdjacencyMatrix, que eran necesarias para manejar las fechas y representar las amistades entre usuarios. En esta etapa también se creó ResourceMeter junto con el método calculateMemory en Photo y Album, para tener desde temprano una forma de medir el consumo de recursos y luego extenderla al resto del programa. Como los archivos de texto todavía no existían, se construyó una primera versión del menú y del carrusel de fotos con álbumes de prueba escritos en el código. Así se pudo avanzar en la interfaz sin esperar a que la carga de datos estuviera lista.

En la etapa de integración se desarrolló User, que reúne los álbumes, amigos y solicitudes de cada persona. A partir de ella se crearon Message y Suggestion, que necesitaban usuarios ya definidos para funcionar. Luego se construyó UdeaBook, la clase central del sistema, que se encarga de la carga de archivos, el login, la búsqueda binaria, los mejores amigos y las sugerencias. Se necesitaba porque debía existir un único punto que coordinara a todos los usuarios y sus relaciones. Después se conectó el menú con UdeaBook para incorporar el login, las solicitudes, las sugerencias y los mensajes. Por último se reemplazaron los datos de prueba por la información cargada desde los archivos, para que el programa trabajara con datos reales que se conservan entre ejecuciones.

En la etapa de cierre se desarrolló un generador de 1000 usuarios, porque los datos hechos a mano eran muy pocos para comprobar que el programa funcionara bien con un volumen grande y que aparecieran todos los casos, como los distintos pesos de los mejores amigos. Con esos datos se aplicó la medición de recursos a todas las funcionalidades, para conocer cuántas iteraciones realiza y cuánta memoria consume cada una, y así detectar las partes que se podían optimizar.

Formato de los archivos de datos

un registro por linea, campos separados por |, usuarios referenciados por userName, fechas aaaa-mm-dd hh:mm (nacimiento sin hora), sin tildes ni ñ. Las rutas de las fotos no se guardan, el programa las arma.

users.txt: primera linea con la cantidad, luego userName|password|email|birthDate|city|country|registrationDate (ordenado por userName).

Ejemplo: ana_gomez|clave123|ana@udea.edu.co|2004-11-02|Medellin|Colombia|2025-01-10 08:30

friendships.txt: userA|userB (cada amistad una sola vez).

Ejemplo: ana_gomez|juan_perez

requests.txt: sender|receiver (solo pendientes).

Ejemplo: juan_perez|laura_rios

albums.txt: owner|albumName|privacy (PUBLIC, PRIVATE o FRIENDS).

Ejemplo: ana_gomez|Vacaciones|FRIENDS

photos.txt: owner|albumName|photoName|captureDate|size|likes

Ejemplo: ana_gomez|Vacaciones|playa01|2025-06-20 16:45|2048|12

messages.txt: sender|receiver|sentDate|read|text (read: 1 o 0; el texto va de ultimo y puede tener |).

Ejemplo: ana_gomez|juan_perez|2025-03-01 10:20|1|Hola!